

Auto increment in MongoDB to store sequence of Unique User ID

```
db.contacts.insert({
  "id":db.contacts.find().count()+1,
  "name":"John Doe",
  "emails":[
    "john@doe.com",
    "john.doe@business.com"
  ],
  "phone":"555111322",
  "status":"Active"
});
```

Aggregation Operations

Aggregation operations process multiple documents and return computed results. You can use aggregation operations to:

- Group values from multiple documents together.
- Perform operations on the grouped data to return a single result.
- Analyze data changes over time.

To perform aggregation operations, you can use:

- [Aggregation pipelines](#)
, which are the preferred method for performing aggregations.
- [Single purpose aggregation methods](#)
, which are simple but lack the capabilities of an aggregation pipeline.

Aggregation Pipelines

An aggregation pipeline consists of one or more [stages](#) that process documents:

- Each stage performs an operation on the input documents. For example, a stage can filter documents, group documents, and calculate values.

- The documents that are output from a stage are passed to the next stage.
- An aggregation pipeline can return results for groups of documents. For example, return the total, average, maximum, and minimum values.

Complete Aggregation Pipeline Examples

```
db.orders.insertMany( [
  { _id: 0, name: "Pepperoni", size: "small", price: 19,
    quantity: 10, date: ISODate( "2021-03-13T08:14:30Z" ) },
  { _id: 1, name: "Pepperoni", size: "medium", price: 20,
    quantity: 20, date : ISODate( "2021-03-13T09:13:24Z" ) },
  { _id: 2, name: "Pepperoni", size: "large", price: 21,
    quantity: 30, date : ISODate( "2021-03-17T09:22:12Z" ) },
  { _id: 3, name: "Cheese", size: "small", price: 12,
    quantity: 15, date : ISODate( "2021-03-13T11:21:39.736Z" ) },
  { _id: 4, name: "Cheese", size: "medium", price: 13,
    quantity: 50, date : ISODate( "2022-01-12T21:23:13.331Z" ) },
  { _id: 5, name: "Cheese", size: "large", price: 14,
    quantity: 10, date : ISODate( "2022-01-12T05:08:13Z" ) },
  { _id: 6, name: "Vegan", size: "small", price: 17,
    quantity: 10, date : ISODate( "2021-01-13T05:08:13Z" ) },
  { _id: 7, name: "Vegan", size: "medium", price: 18,
    quantity: 10, date : ISODate( "2021-01-13T05:10:13Z" ) }
])
```

_id	name	size	price	quantity	date
0	Pepperoni	small	19	10	ISODate("2021-03-13T08:14:30Z")
1	Pepperoni	medium	20	20	ISODate("2021-03-13T09:13:24Z")
2	Pepperoni	large	21	30	ISODate("2021-03-17T09:22:12Z")
3	Cheese	small	12	15	ISODate("2021-03-13T11:21:39Z")
4	Cheese	medium	13	50	ISODate("2022-01-12T21:23:13.331Z")
5	Cheese	large	14	10	ISODate("2022-01-12T05:08:13Z")
6	Vegan	small	17	10	ISODate("2022-01-13T05:08:13Z")

_id	name	size	price	quantity	date
7	Vegan	medium	18	10	ISODate("2021-01-13T05:10:13Z")

Calculate Total Order Quantity

The following aggregation pipeline example contains two [stages](#) and returns the total order quantity of medium size pizzas grouped by pizza name:

```
db.orders.aggregate( [
    // Stage 1: Filter pizza order documents by pizza size
    {
        $match: { size: "medium" }
    },
    // Stage 2: Group remaining documents by pizza name and
    // calculate total quantity
    {
        $group: { _id: "$name", totalQuantity: { $sum: "$quantity" } }
    }
] )
```

IT will no print id because `_id:0`

```
db.orders.aggregate([
    $group: { _id: "$name", count: { $sum: 1 } }
], {$project: {_id: 0}}]);
```

It will print the id as name

```
db.orders.aggregate([  
  $group: { _id: "$name", count: { $sum: 1 } }  
], {$project: { _id: 1, count: 1 } });
```

In a `$group` stage, `_id` is used to designate the group condition. You obviously need it.

If you're familiar with the SQL world, think of it as the `GROUP BY` clause.

```
SELECT SUM() FROM table GROUP BY name).
```

\$unwind Deconstructs an array field from the input documents to output a document for each element. Each output document is the input document with the value of the array field replaced by the element.

```
{  
  "_id" : 1,  
  "shirt" : "Half Sleeve",  
  "sizes" : [  
    "medium",  
    "XL",  
    "free"  
  ]  
}
```

```
db.test1.aggregate( [ { $unwind : "$sizes" } ] );
```

```
{ "_id" : 1, "shirt" : "Half Sleeve", "sizes" : "medium" }  
{ "_id" : 1, "shirt" : "Half Sleeve", "sizes" : "XL" }  
{ "_id" : 1, "shirt" : "Half Sleeve", "sizes" : "free" }
```

Ex:2

```
db.article.aggregate(
```

```
{ $project : {
  author : 1,
  title : 1,
  tags : 1
}},
{ $unwind : "$tags" }
);
```

```
create table #article (author varchar(50), title varchar(50))
insert into #article values('bob', 'this is my title')

create table #tags(tags varchar(10))
insert into #tags values('fun')
insert into #tags values('good')
insert into #tags values('fun')

SELECT author, title, tags
FROM #article, #tags
```

100 %

Results Messages

	author	title	tags
1	bob	this is my title	fun
2	bob	this is my title	good
3	bob	this is my title	fun

tags:['fun','good','fun']

Output:(JSON format)

```
{ "author": "bob", "title": "this is my title", "tags": "fun"},
{ "author": "bob", "title": "this is my title", "tags": "good"},
{ "author": "bob", "title": "this is my title", "tags": "fun"}
```

The **\$match** stage:

- Filters the pizza order documents to pizzas with a size of medium.
- Passes the remaining documents to the **\$group** stage.

The **\$group** stage:

- Groups the remaining documents by pizza name.
- Uses `$sum` to calculate the total order quantity for each pizza name. The total is stored in the `totalQuantity` field returned by the aggregation pipeline.

Example output:

[
{ <code>_id</code> : 'Cheese', <code>totalQuantity</code> : 50 },
{ <code>_id</code> : 'Vegan', <code>totalQuantity</code> : 10 },
{ <code>_id</code> : 'Pepperoni', <code>totalQuantity</code> : 20 }
]

Calculate Total Order Value and Average Order Quantity

The following example calculates the total pizza order value and average order quantity between two dates:

```
db.orders.aggregate( [
```

```
  // Stage 1: Filter pizza order documents by date range
```

```
{
```

```
  $match:
```

```
{
```

```
    "date": { $gte: new ISODate( "2020-01-30" ), $lt: new
    ISODate( "2022-01-30" ) }
```

```
}
```

```
},
```

```
  // Stage 2: Group remaining documents by date and calculate
  results
```

```

{
  $group:
  {
    _id: { $dateToString: { format: "%Y-%m-%d", date: "$date" } },
    totalOrderValue: { $sum: { $multiply: [ "$price", "$quantity" ] } },
    averageOrderQuantity: { $avg: "$quantity" }
  }
},
// Stage 3: Sort documents by totalOrderValue in descending
order
{
  $sort: { totalOrderValue: -1 }
}
])

```

The `$match` stage:

- Filters the pizza order documents to those in a date range specified using `$gte` and `$lt`
- Passes the remaining documents to the `$group` stage.

The `$group` stage:

- Groups the documents by date using `$dateToString`
- For each group, calculates:
 - Total order value using `$sum` and `$multiply`
 - Average order quantity using `$avg`

- Passes the grouped documents to the `$sort` stage.

The `$sort` stage:

- Sorts the documents by the total order value for each group in descending order (-1).
- Returns the sorted documents.

Example output:

[
{ _id: '2022-01-12', totalOrderValue: 790, averageOrderQuantity: 30
{ _id: '2021-03-13', totalOrderValue: 770, averageOrderQuantity: 15
{ _id: '2021-03-17', totalOrderValue: 630, averageOrderQuantity: 30
{ _id: '2021-01-13', totalOrderValue: 350, averageOrderQuantity: 10
]

Method	Description
db.collection.estimatedDocumentCount()	Returns an approximate count of the documents in a collection or a view.
db.collection.count()	Returns a count of the number of documents in a collection or a view.
db.collection.distinct()	Returns an array of documents that have distinct values for the specified field.

Update Documents Using an Aggregation Pipeline

To update documents with an aggregation pipeline, use:

Command	mongosh Methods
findAndModify	db.collection.findOneAndUpdate() db.collection.findAndModify()
update	db.collection.updateOne() db.collection.updateMany() Bulk.find.update() Bulk.find.updateOne() Bulk.find.upsert()

Use an Aggregation Pipeline for Updates

Starting in MongoDB 4.2,

[db.collection.findOneAndUpdate\(\)](#)

can accept an aggregation pipeline for the update. The pipeline can consist of the following stages:

- [\\$addFields](#) and its alias [\\$set](#)
- [\\$project](#) and its alias [\\$unset](#)
- [\\$replaceRoot](#) and its alias [\\$replaceWith](#)

Using the aggregation pipeline allows for a more expressive update statement, such as expressing conditional updates based on current field values or updating one field using the value of another field(s).

For example, create a collection `students2` with the following documents:

```
db.students2.insertMany( [
  {
    "_id" : 1,
    "grades" : [
      { "grade" : 80, "mean" : 75, "std" : 6 },
      { "grade" : 85, "mean" : 90, "std" : 4 },
```

```

        { "grade" : 85, "mean" : 85, "std" : 6 }
    ]
},
{
    "_id" : 2,
    "grades" : [
        { "grade" : 90, "mean" : 75, "std" : 6 },
        { "grade" : 87, "mean" : 90, "std" : 3 },
        { "grade" : 85, "mean" : 85, "std" : 4 }
    ]
}
])

```

The following operation finds a document where the `_id` field equals 1 and uses an aggregation pipeline to calculate a new field `total` from the `grades` field:

findOneAndUpdate

```

db.students2.findOneAndUpdate(

    { _id : 1 },

    [ { $set: { "total" : { $sum: "$grades.grade" } } } ],

    // The $set stage is an alias for ``$addField`` stage

    { returnNewDocument: true }

)

```

The operation returns the updated document :

```

{
    "_id" : 1,

```

```
"grades" : [ { "grade" : 80, "mean" : 75, "std" : 6 }, { "grade" : 85,
"mean" : 90, "std" : 4 }, { "grade" : 85, "mean" : 85, "std" : 6 } ],

"total" : 250

}
```

findAndModify()

```
db.students2.insertMany( [
  {
    "_id" : 1,
    "grades" : [
      { "grade" : 80, "mean" : 75, "std" : 6 },
      { "grade" : 85, "mean" : 90, "std" : 4 },
      { "grade" : 85, "mean" : 85, "std" : 6 }
    ]
  },
  {
    "_id" : 2,
    "grades" : [
      { "grade" : 90, "mean" : 75, "std" : 6 },
      { "grade" : 87, "mean" : 90, "std" : 3 },
      { "grade" : 85, "mean" : 85, "std" : 4 }
    ]
  }
] )
```

```
db.students2.findAndModify( {
  query: { "_id" : 1 },
```

```
    update: [ { $set: { "total" : { $sum: "$grades.grade" } } } ],  
// The $set stage is an alias for ``$addField`` stage  
  
    new: true  
  
  } )
```

Output:

```
{  
  "_id" : 1,  
  
  "grades" : [ { "grade" : 80, "mean" : 75, "std" : 6 }, { "grade" : 85,  
"mean" : 90, "std" : 4 }, { "grade" : 85, "mean" : 85, "std" : 6 } ],  
  
  "total" : 250  
  
}
```

Use Variables in let

To define variables that you can access elsewhere in the command, use the

let

option.

```
db.cakeFlavors.insertMany( [  
  { _id: 1, flavor: "chocolate" },  
  { _id: 2, flavor: "strawberry" },  
  { _id: 3, flavor: "cherry" }  
])
```

The following example defines a `targetFlavor` variable in `let` and uses the variable to change the cake flavor from cherry to orange:

```

db.cakeFlavors.findAndModify( {
  query: {
    $expr: { $eq: [ "$flavor", "$$targetFlavor" ] }
  },
  update: { flavor: "orange" },
  let: { targetFlavor: "cherry" }
})

```

Aggregation pipeline - updateOne()

```

db.students3.insertMany( [

  { "_id" : 1, "tests" : [ 95, 92, 90 ], "average" : 92, "grade" : "A",
    "lastUpdate" : ISODate("2020-01-23T05:18:40.013Z") },

  { "_id" : 2, "tests" : [ 94, 88, 90 ], "average" : 91, "grade" : "A",
    "lastUpdate" : ISODate("2020-01-23T05:18:40.013Z") },

  { "_id" : 3, "tests" : [ 70, 75, 82 ], "lastUpdate" :
    ISODate("2019-01-01T00:00:00Z") }

] )

db.students3.updateOne(
  { _id: 3 },
  [
    { $set: { average: { $trunc: [ { $avg: "$tests" }, 0 ] }, lastUpdate:
      "$$NOW" } },
    { $set: { grade: { $switch: {
      branches: [
        { case: { $gte: [ "$average", 90 ] }, then: "A" },
        { case: { $gte: [ "$average", 80 ] }, then: "B" },
        { case: { $gte: [ "$average", 70 ] }, then: "C" },
        { case: { $gte: [ "$average", 60 ] }, then: "D" }
      ]
    } } } }
  ]
)

```

```

    ],
    default: "F"
  } } } }
})

```

The third document `_id: 3` is missing the `average` and `grade` fields. Using an aggregation pipeline, you can update the document with the calculated grade average and letter grade.

`$trunc`

truncates a number to a whole integer or to a specified decimal place.

First Stage

The `$set` stage:

- calculates a new field `average` based on the average of the `tests` field. See `$avg` for more information on the `$avg` aggregation operator and `$trunc` for more information on the `$trunc` truncate aggregation operator.
- sets the field `lastUpdate` to the value of the aggregation variable `NOW`. The aggregation variable `NOW` resolves to the current datetime value and remains the same throughout the pipeline. To access aggregation variables, prefix the variable with double dollar signs `$$` and enclose in quotes.

Second Stage

The `$set` stage calculates a new field `grade` based on the `average` field calculated in the previous stage. See `$switch` for more information on the `$switch` aggregation operator.

MongoDB Indexes

- What indexes are and why you need to use them.

- How to select fields to index.
- Understand various types of indexes in MongoDB.
- How to evaluate the index usage.
- How to manage indexes such as creating and removing them.

Indexes

Indexes support the efficient execution of queries in MongoDB. Without indexes, MongoDB must perform a collection scan, i.e. scan every document in a collection, to select those documents that match the query statement. If an appropriate index exists for a query, MongoDB can use the index to limit the number of documents it must inspect.

Default `_id` Index

MongoDB creates a [unique index](#) on the `_id` field during the creation of a collection. The `_id` index prevents clients from inserting two documents with the same value for the `_id` field. You cannot drop this index on the `_id` field.

Create an Index

```
db.COLLECTION_NAME.createIndex({KEY:1})
```

The key determines the field on the basis of which you want to create an index and 1 (or -1) determines the order in which these indexes will be arranged(ascending or descending).

Example –

```
db.mycol.createIndex({"age":1})  
{  
  "createdCollectionAutomatically" : false,
```

```
"numIndexesBefore" : 1,  
"numIndexesAfter" : 2,  
"ok" : 1  
}
```

Drop an index:

In order to drop an index, MongoDB provides the `dropIndex()` method.

Syntax –

```
db.NAME_OF_COLLECTION.dropIndex( {KEY:1} )
```

The `dropIndex()` methods can only delete one index at a time. In order to delete (or drop) multiple indexes from the collection, MongoDB provides the `dropIndexes()` method that takes multiple indexes as its parameters.

Syntax –

```
db.NAME_OF_COLLECTION.dropIndexes( {KEY1:1, KEY2,  
1} )
```

The `dropIndex()` methods can only delete one index at a time. In order to delete (or drop) multiple indexes from the collection, MongoDB provides the `dropIndexes()` method that takes multiple indexes as its parameters.

Get description of all indexes :

The `getIndexes()` method in MongoDB gives a description of all the indexes that exists in the given collection.

Syntax –

```
db.NAME_OF_COLLECTION.getIndexes( )
```

It will retrieve all the description of the indexes created within the collection.

The following fields returned in this output are particularly relevant to understanding how indexes work:

- **winningPlan**: This document within the **queryPlanner** section describes how MongoDB decided to execute the query. Depending on the query type, the detailed structure of the **winningPlan** may differ, but here the key thing to note is **COLLSCAN**. The presence of this value means that MongoDB needed to go through the full collection without any aids to find the requested documents.
- **nReturned**: This value tells you how many documents were returned by a given query. Here, just a single mountain peak matches the query.
- **executionTimeMillis**: This value represents the execution time. With such a small collection, its importance is negligible. However, when analyzing the performance of queries against larger or more complex collections, it is an important metric to keep in mind.
- **totalKeysExamined**: This tells you how many **index** entries MongoDB checked to find the requested documents. Because the collection scan was used and you haven't created any indexes yet, the value is **0**.
- **totalDocsExamined**: This value indicates how many documents MongoDB had to read from the collection. Because MongoDB performed a collection scan, its value is **5**, the total count of all documents in the collection. The larger the collection, the bigger the value in this field when indexes are not used.

```
db.peaks.createIndex( { "height": 1 } )
```

Output

```
{
  "createdCollectionAutomatically" : false,
  "numIndexesBefore" : 1,
  "numIndexesAfter" : 2,
  "ok" : 1
}
```

```
db.peaks.find(
  { "height": { $gt: 8700 } }
).explain("executionStats")
```

Step 3 — Creating Unique Indexes

In MongoDB, it's impossible to insert two documents into the collection if they both have the same `_id` values. This is because the database automatically maintains a single-field index on the `_id` field that, in addition to helping speed up document lookups, ensures the uniqueness of the `_id` field value. This step explains how you can create indexes to ensure the values of a given field will be unique for every document in a collection.

To illustrate, run the following `createIndex()` method. This command's syntax is similar to the one used in the previous step except, this time, a second parameter is passed to `createIndex()` with additional settings for the index. The `{ "unique": true }` indicates that the created index will ensure that the values of the specified field (`name`) can't repeat:

```
db.peaks.createIndex( { "name": 1 }, { "unique": true } )
```

Output

```
{
  "createdCollectionAutomatically" : false,
  "numIndexesBefore" : 2,
  "numIndexesAfter" : 3,
  "ok" : 1
}
```

```
db.peaks.insertOne({
  "name": "Everest",
  "height": 9200,
  "location": ["India"],
  "ascents": {
    "first": {
      "year": 2020
    },
    "first_winter": {
      "year": 2021
    },
    "total": 2
  }
})
```

MongoServerError: E11000 duplicate key error collection: Day6.peaks index: name_1 dup key: { name: "Everest" }

Step 4 — Creating an Index on an Embedded Field

Whenever you query a collection using a field within a nested document that doesn't have an index, MongoDB not only has to retrieve all documents from the collection, but it must also traverse each nested document.

As an example, run the following query. This will return any documents whose `total` — a field nested within the `ascents` document found in each document in the `peaks` collection — is greater than 300 and sorts the results in descending order:

```
db.peaks.find(  
  { "ascents.total": { $gt: 300 } }  
).sort({ "ascents.total": -1 })
```

Output:

```
{ "_id" : ObjectId("61212a8300c8304536a86b2f"), "name" :  
"Everest", "height" : 8848, "location" : [ "Nepal",  
"China" ], "ascents" : { "first" : { "year" : 1953 },  
"first_winter" : { "year" : 1980 }, "total" : 5656 } }  
  
{ "_id" : ObjectId("61212a8300c8304536a86b32"), "name" :  
"Lhotse", "height" : 8516, "location" : [ "Nepal",  
"China" ], "ascents" : { "first" : { "year" : 1956 },  
"first_winter" : { "year" : 1988 }, "total" : 461 } }  
  
{ "_id" : ObjectId("61212a8300c8304536a86b33"), "name" :  
"Makalu", "height" : 8485, "location" : [ "China",  
"Nepal" ], "ascents" : { "first" : { "year" : 1955 },  
"first_winter" : { "year" : 2009 }, "total" : 361 } }  
  
{ "_id" : ObjectId("61212a8300c8304536a86b30"), "name" :  
"K2", "height" : 8611, "location" : [ "Pakistan", "China"
```

```
], "ascents" : { "first" : { "year" : 1954 },
"first_winter" : { "year" : 1921 }, "total" : 306 } }
```

```
db.peaks.find(
  { "ascents.total": { $gt: 300 } }
).sort({ "ascents.total": -1 }).explain("executionStats")
```

```
db.peaks.createIndex( { "ascents.total": 1 } )
```

```
db.peaks.find(
  { "ascents.total": { $gt: 300 } }
).sort({ "ascents.total": -1 }).explain("executionStats")
```

Output

```
{
  "queryPlanner" : {
    "winningPlan" : {
      "inputStage" : {
        "stage" : "IXSCAN",
        "indexName" :
"ascents.total_-1",
        . . .
      }
    },
    . . .
  },
  "executionStats" : {
    "nReturned" : 4,
    "executionTimeMillis" : 0,
    "totalKeysExamined" : 4,
    "totalDocsExamined" : 4,
    "direction" : "backward",
    . . .
  },
  . . .
}
```

Step 5 — Creating a Compound Field Index

The examples so far in this guide are helpful for understanding the benefits of using indexes, but document filtering queries used in real world applications are rarely this simple. This step explains how MongoDB uses indexes when executing queries on more than one field and how to use compound field indexes to target such queries specifically.

Recall from Step 2 when you created a single field index on the `height` field in order to more efficiently query the `peaks` collection to find the highest mountain peaks. With this index in place, let's analyze how MongoDB will perform a similar but slightly more complex query. Try finding mountains with a height of less than 8600 meters whose first winter ascent occurred after the year 1990:

```
db.peaks.find(  
  {  
    "ascents.first_winter.year": { $gt: 1990 },  
    "height": { $lt: 8600 }  
  }  
) .sort({ "height": -1 })
```

```
db.peaks.find(  
  {  
    "ascents.first_winter.year": { $gt: 1990 },  
    "height": { $lt: 8600 }
```

```
}
```

```
).sort({ "height": -1 }).explain("executionStats")
```

Output

```
{
  "queryPlanner" : {
    "winningPlan" : {
      "stage" : "IXSCAN",
      "indexName" : "height_1",
      . . .
    },
    . . .
  },
  "executionStats" : {
    "nReturned" : 1,
    "executionTimeMillis" : 0,
    "totalKeysExamined" : 3,
    "totalDocsExamined" : 3,
    . . .
  },
  . . .
}
```

```
db.peaks.createIndex(
  {
    "ascents.first_winter.year": 1,
    "height": -1
  }
)
```

This time the query again used an index scan, but the index is different. Now, the `ascents.first_winter.year_1_height_-1` index that you just created is chosen over the previously-used `height_1` index:

Delete the height index first then it will display the output like below

$\{$

}

)

```
{ "_id" : ObjectId("61212a8300c8304536a86b2f"), "name" :
"Everest", "height" : 8848, "location" : [ "Nepal",
"China" ], "ascents" : { "first" : { "year" : 1953 },
"first_winter" : { "year" : 1980 }, "total" : 5656 } }
{ "_id" : ObjectId("61212a8300c8304536a86b31"), "name" :
"Kangchenjunga", "height" : 8586, "location" : [ "Nepal",
"India" ], "ascents" : { "first" : { "year" : 1955 },
"first_winter" : { "year" : 1986 }, "total" : 283 } }

{ "_id" : ObjectId("61212a8300c8304536a86b32"), "name" :
"Lhotse", "height" : 8516, "location" : [ "Nepal",
"China" ], "ascents" : { "first" : { "year" : 1956 },
"first_winter" : { "year" : 1988 }, "total" : 461 } }

{ "_id" : ObjectId("61212a8300c8304536a86b33"), "name" :
"Makalu", "height" : 8485, "location" : [ "China",
"Nepal" ], "ascents" : { "first" : { "year" : 1955 },
"first_winter" : { "year" : 2009 }, "total" : 361 } }
```

Notice that none of these peaks are only in Nepal. Each of these four peaks span more than one country as indicated by their `location` fields, all of which are an array of multiple values. What is more, these values can appear in different orders. For example, Lhotse is listed as being in ["Nepal", "China"], whereas Makalu is listed as being in ["China", "Nepal"].

Because there is no index available spanning the `location` field, MongoDB currently does a full collection scan to execute that query. Let's create a new index for the `location` field:

```
db.peaks.createIndex( { "location": 1 } )
```

Output

```
{
  "createdCollectionAutomatically" : false,
  "numIndexesBefore" : 5,
  "numIndexesAfter" : 6,
  "ok" : 1
}
```

```
db.peaks.find(
  { "location": "Nepal" }
).explain("executionStats")
```

Output

```
{
  "queryPlanner" : {
    "winningPlan" : {
      "inputStage" : {
        "stage" : "IXSCAN",
        "indexName" :
"location_1",
        "isMultiKey" : true,
        . . .
      },
      . . .
    },
    "executionStats" : {
      "nReturned" : 4,
      "executionTimeMillis" : 0,
      "totalKeysExamined" : 4,
      "totalDocsExamined" : 4,
      . . .
    }
  }
}
```

The number of returned documents matches the total number of examined index keys and examined documents. This means that the index was used as the sole source of information for the query. How was that possible if the field values are arrays of more than one value, and the query asked for mountains with one of the locations matching Nepal?

Notice the `isMultiKey` property listed as `true` in the output. MongoDB automatically created a multi-key index for the `location` field. If you create an index for a field holding arrays, MongoDB automatically determines it needs to create a multi-key index and creates separate index entries for every element of these arrays.

So, for a document that has a `location` field storing the array `[ "China", "Nepal" ]`, two separate index entries appear for the same document, one for China and another for Nepal. This way, MongoDB can use the index efficiently even if the query requests a partial match against the array contents.

Step 7 — Listing and Removing Indexes on a Collection

In the previous steps, you've learned how to create different types of indexes. When the database grows or requirements change, it's important to be able to know what indexes are defined and sometimes remove unwanted ones. Indexes that are no longer useful can have a negative impact on the database's performance, since MongoDB must still maintain them any time you add or change data.

To list all the indexes you've defined on the `peaks` collection throughout this tutorial, you can use the `getIndexes()` method:

`db.peaks.getIndexes()`

Output

```
[
  {
    "v" : 2,
    "key" : {
      "_id" : 1
    },
    "name" : "_id_"
  },
  {
    "v" : 2,
    "key" : {
      "height" : 1
    },
    "name" : "height_1"
  },
  {
    "v" : 2,
```

```

        "unique" : true,
        "key" : {
            "name" : 1
        },
        "name" : "name_1"
    },
    {
        "v" : 2,
        "key" : {
            "ascents.total" : 1
        },
        "name" : "ascents.total_1"
    },
    {
        "v" : 2,
        "key" : {
            "ascents.first_winter.year" : 1,
            "height" : -1
        },
        "name" :
"ascents.first_winter.year_1_height_-1"
    },
    {
        "v" : 2,
        "key" : {
            "location" : 1
        },
        "name" : "location_1"
    }
}
]

```

To delete an existing index, you can use either of these properties with the `dropIndex()` method. The following example will delete the `height_1` index by using the definition of its contents:

```
db.peaks.dropIndex( { "height": 1 } )
```

Since the `{ "height": 1 }` matches the single field index on `height` named `height_1`, MongoDB will remove that index and reply with a success message indicating how many indexes there were prior to removing this one:

Output

```
{ "nIndexesWas" : 6, "ok" : 1 }
```

```
db.peaks.dropIndex("ascents.first_winter.year_1_height_-1")
```

Output

```
{ "nIndexesWas" : 5, "ok" : 1 }
```

You can confirm that these two indexes have been indeed removed from the list of collection indexes by calling `getIndexes()` again:

```
db.peaks.getIndexes()
```

Output

```
[
  {
    "v" : 2,
    "key" : {
      "_id" : 1
    },
    "name" : "_id_"
  },
  {
    "v" : 2,
    "unique" : true,
    "key" : {
      "name" : 1
    },
    "name" : "name_1"
  },
  {
    "v" : 2,
    "key" : {
      "ascents.total" : 1
    },
    "name" : "ascents.total_1"
  },
  {
    "v" : 2,
    "key" : {
      "location" : 1
    },
    "name" : "location_1"
  }
]
```

MongoDB can use the multikey index to look up the first element of the query array but cannot use the multikey index scan to find the whole array. Instead, after using the multikey index to look up the first element of the query array, MongoDB retrieves the associated documents and filters for documents whose array matches the array in the query.